

NatScript Language and NatVM Architecture Proposal

Status

Conceptual design document. Intended for future implementation after the core NatOS operating system and VM architecture have matured.

1. Purpose

NatScript is envisioned as the future high-level programming language for the NatOS ecosystem.

The purpose of NatScript is not simply to create another general-purpose programming language. Its purpose is to make programming **physical computers** intuitive.

NatOS is intended to provide the underlying operating environment, hardware abstraction, system services, application isolation, and execution environment. NatScript would provide the human-friendly interface through which developers create applications running on that environment.

The intended architecture is:

```
NatScript Source
    ↓
NatScript Compiler
    ↓
NatOS Bytecode
    ↓
NatVM
    ↓
NatOS System Services
    ↓
Physical Hardware
```

The developer should be able to program the behavior of a physical device without needing to understand the specific microcontroller, peripheral registers, communication buses, or low-level implementation underneath it.

2. Existing Foundation

NatOS already contains the beginnings of the necessary execution architecture.

The current system uses a custom bytecode instruction set executed by a virtual machine. Applications are therefore separated from the ESP32's native instruction set.

This establishes an important distinction:

```
ESP32 machine instructions
    ≠
NatOS bytecode
    ≠
Future NatScript source
```

NatScript should compile to the existing NatOS bytecode rather than directly to ESP32 machine code.

The existing VM therefore becomes the initial compilation target for NatScript.

The current architecture should be studied and stabilized before major language design work begins.

3. NatVM Concept

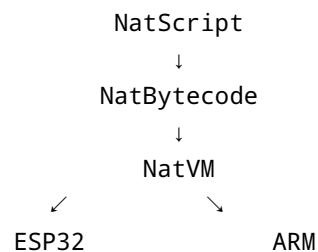
The NatOS virtual machine can be viewed conceptually as a small virtual processor.

NatVM defines its own:

- instruction set
- registers or virtual registers
- memory model
- control flow
- function/call semantics
- system-call mechanism
- application boundaries
- execution rules

The physical ESP32 executes the VM implementation, while the application executes within the VM's abstraction.

This provides a potential portability layer:





In the long term, the same NatBytecode could potentially execute on different physical processors if a compatible NatVM implementation exists.

The goal is therefore not necessarily to create a new physical processor. NatVM is initially a **virtual processor architecture implemented in software**.

A future hardware implementation of the instruction set could theoretically be explored, but this is outside the initial scope.

4. NatScript Design Philosophy

NatScript should not attempt to become another Java, C++, or Python.

Its design should begin with a different question:

What concepts does a programmer actually need when programming a physical machine?

Physical systems continuously react to their environment. They receive sensor input, execute control logic, communicate with other devices, and alter the physical world through actuators.

Consequently, NatScript should make these behaviors natural to express.

The language should prioritize:

- devices
- events
- recurring tasks
- state
- functions
- data
- system services
- communication
- hardware capabilities

The language should not require object-oriented programming as its fundamental paradigm.

5. Event-Oriented Syntax

One potentially important feature is explicit event-oriented syntax.

For example:

```
when scanner.scanned(code) {  
    inventory.process(code);  
}
```

This expresses:

Whenever this event occurs, execute this behavior.

The underlying implementation could use an event queue, interrupt mechanism, scheduler, or other NatOS service without exposing those mechanisms to the developer.

Another example:

```
when button.pressed {  
    display.show("Hello");  
}
```

Or:

```
when wifi.connected {  
    synchronize_inventory();  
}
```

The `when` construct should eventually become a first-class language feature if the underlying NatOS event architecture supports it cleanly.

6. Recurring Tasks

Physical systems frequently require work to happen at a recurring interval.

NatScript could make this explicit:

```
every 10ms {  
    balance.update();  
}
```

or:

```
every 1s {  
    sensors.sample();  
}
```

This is preferable to forcing developers to manually construct timing loops for common use cases.

The compiler could translate the construct into an appropriate NatOS scheduling mechanism.

Conceptually:

```
every 10ms {  
    balance.update();  
}
```

could become something similar to:

```
REGISTER_TASK  
INTERVAL 10  
CALL balance.update  
YIELD
```

The actual implementation should depend on the capabilities and guarantees of the NatOS scheduler.

7. Physical-System Programming

The language should make physical behavior readable.

For example, a hypothetical robot application might eventually look like:

```
every 5ms {  
    orientation = imu.read();  
    balance.update(orientation);  
}  
  
when obstacle.detected {  
    movement.stop();  
}  
  
when robot.fallen {  
    recovery.begin();  
}
```

```
when voice.heard(command) {  
    process_command(command);  
}
```

This resembles the actual behavioral architecture of a robot:

- continuously maintain balance
- react to obstacles
- recover from failure states
- respond to human interaction

The program becomes almost a behavioral specification of the machine.

8. Hardware as First-Class Concepts

NatScript should ideally allow hardware capabilities to be represented directly.

Possible future APIs could include:

```
scanner.read();  
display.show(data);  
imu.read();  
motor.set_position(45);  
camera.capture();  
speaker.say("Hello");  
bluetooth.send(data);  
wifi.request(url);  
battery.level();  
gpio.write(pin, value);
```

The exact syntax is intentionally undecided.

The important principle is that the application should interact with **capabilities**, not implementation details.

A developer should think:

"I have a scanner."

rather than:

"I have a peripheral attached through this particular bus using these particular registers."

NatOS should handle the underlying implementation.

9. Example: Inventory Scanner

A future NatScript application might resemble:

```
device scanner;
device display;
device wifi;

when scanner.scanned(code) {
    product = inventory.lookup(code);

    if product.exists {
        display.show(product);
        wifi.post("/inventory/update", product);
    }
}
```

The customer would therefore be able to create an application specifically suited to their own warehouse or inventory system.

The physical scanner becomes a programmable platform rather than a device locked to one manufacturer's software.

10. Example: Robotics

The same programming model could eventually apply to robotic systems.

For example:

```
device imu;
device left_leg;
device right_leg;

every 5ms {
    orientation = imu.read();

    balance.update(
        orientation,
        left_leg,
        right_leg
    );
}

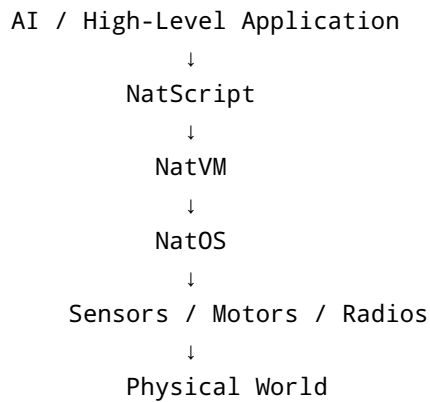
when robot.fallen {
```

```
    recovery.begin();  
}
```

The application does not need to know whether the motors communicate through PWM, SPI, CAN, or another mechanism.

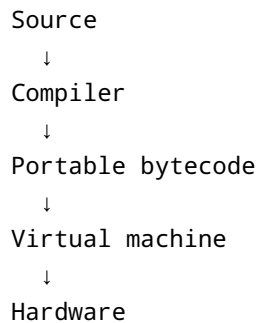
NatOS provides the underlying device abstraction.

This creates the intended relationship:



11. Relationship to Java and Other VM Languages

NatScript would share an important architectural concept with languages such as Java:



However, NatScript's target domain would be substantially different.

Java's JVM is a general-purpose application runtime.

NatVM would be designed around constrained physical computers and potentially robotics.

NatScript should therefore prioritize:

- constrained memory
- low overhead
- predictable execution
- physical I/O
- sensors
- actuators
- networking
- power management
- event-driven behavior
- application isolation
- hardware abstraction

The goal is not to recreate Java.

The goal is to explore whether the virtual-machine model can be designed specifically for physical computing.

12. Programming Paradigm

NatScript should initially favor procedural, event-driven, and data-oriented programming.

Object-oriented programming may be supported where useful, but should not become a requirement for ordinary applications.

A physical system can often be described more naturally as:

```
devices
state
events
tasks
functions
data
```

rather than an elaborate hierarchy of objects.

For example:

```
device imu;
device motor;
device camera;

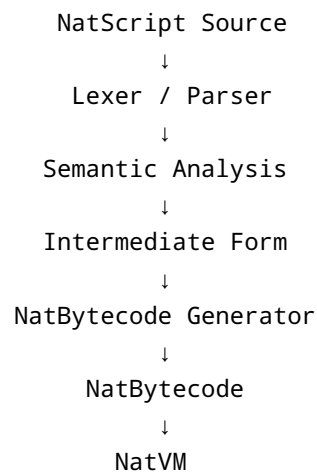
every 10ms {
    update_balance();
}
```

```
}  
  
when obstacle.detected {  
    motor.stop();  
}
```

This may be more appropriate for embedded and robotic applications than forcing every component into a class hierarchy.

13. Compiler Architecture

The intended NatScript toolchain is:



The compiler should eventually provide useful diagnostics before deployment.

For example:

```
error: motor.position() requires a motor-capable device  
warning: task "balance" has no declared timing budget  
error: network permission not granted
```

This would allow the language and operating system to work together rather than treating the OS as a passive runtime.

14. Security and Permissions

Because NatScript applications may eventually control physical hardware and communicate with customer servers, application permissions should be considered early in the architecture.

Possible permissions include:

```
network
bluetooth
storage
scanner
camera
microphone
motor
gpio
system
```

An application might declare:

```
permissions {
    scanner;
    network;
    display;
}
```

NatOS could then restrict applications from accessing capabilities they were not granted.

This becomes increasingly important for robotics, industrial equipment, and network-connected devices.

15. What Should Not Be Done Yet

NatScript should not be implemented immediately merely because the concept is exciting.

Before language development begins, the existing NatOS VM should be studied thoroughly.

The following should be documented first:

- every opcode
- instruction encoding
- register behavior
- memory model
- call conventions
- stack behavior
- syscall ABI
- application lifecycle
- error behavior
- scheduling behavior
- memory boundaries
- performance characteristics

Then determine whether the current VM is a suitable compiler target.

If the VM is inadequate, improve the VM before designing NatScript syntax around it.

A beautiful programming language targeting an awkward virtual machine would simply move the problem one layer upward.

16. Development Strategy

The recommended sequence is:

Phase 1: Understand

Learn the existing NatOS assembly language and VM completely.

Write small programs manually.

Understand exactly how an application executes.

Phase 2: Stabilize

Document the NatVM instruction set and application ABI.

Identify inconsistencies and unnecessary complexity.

Phase 3: Improve the Application Interface

Develop clean system APIs for common device capabilities.

Phase 4: Prototype NatScript

Create a minimal language containing:

- variables
- expressions
- functions
- conditionals
- loops
- events
- recurring tasks
- system/device calls

Phase 5: Compile to Existing NatBytecode

Make NatScript compile into the existing VM instruction set.

Phase 6: Build Real Applications

Use NatScript to build:

- scanner applications
- Bluetooth applications
- sensor applications
- networking applications
- device-control applications

Phase 7: Robotics

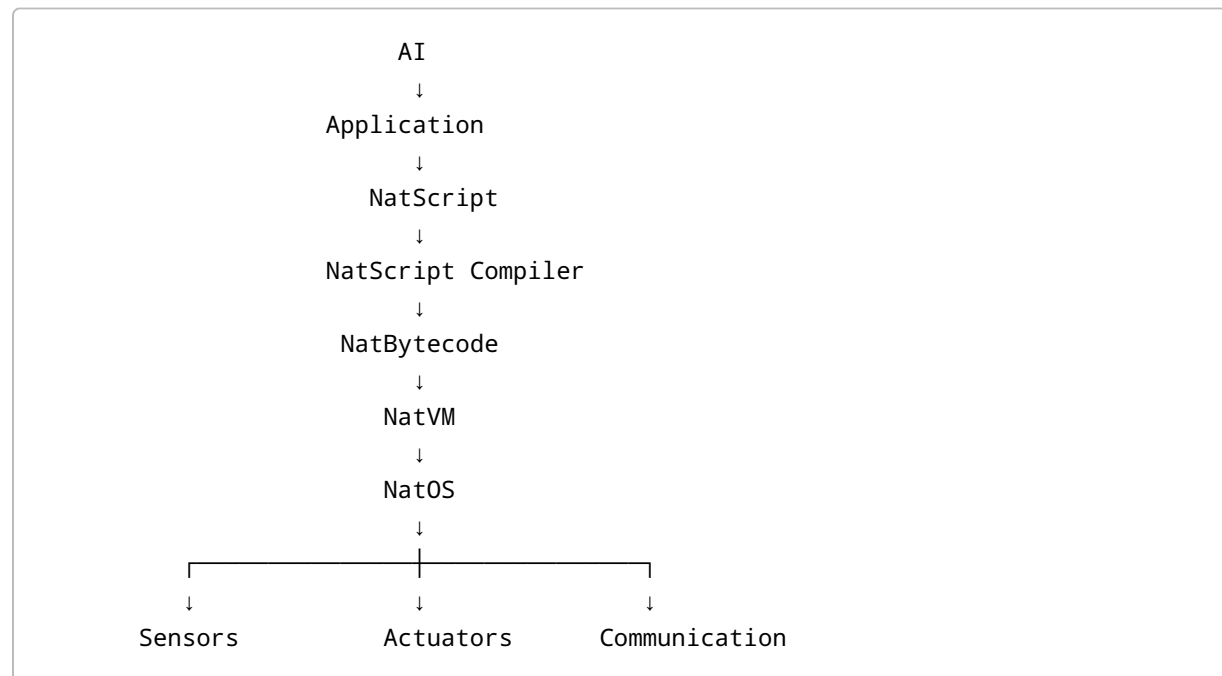
Extend the device abstraction toward:

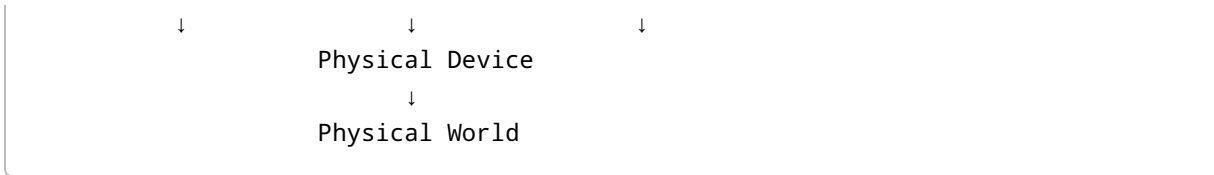
- IMUs
- motors
- encoders
- cameras
- microphones
- speakers
- power systems
- recovery behaviors

Only after these layers work together should the language become significantly more ambitious.

17. Long-Term Vision

The ultimate architecture envisioned for the NatOS ecosystem is:





The broader objective is to provide a reusable software foundation for physical computing.

Instead of every developer repeatedly building:

```
hardware initialization
drivers
memory management
networking
device interfaces
application infrastructure
diagnostics
```

they can build on an existing platform.

The developer's work becomes increasingly focused on the actual application:

What should this physical computer do?

rather than:

How do I rebuild the computer underneath it?

18. Core Design Principle

The central principle of NatScript should be:

Make physical computing feel like programming a computer, rather than programming a collection of electrical components.

NatOS provides the infrastructure.

NatVM provides the execution environment.

NatBytecode provides the portable intermediate representation.

NatScript provides the human-facing programming language.

Together, these components could eventually allow developers to build applications for scanners, sensors, industrial devices, embedded computers, and robots using a common conceptual platform.

The immediate goal is not to prove the entire vision.

The immediate goal is to make one small physical computer demonstrate that the abstraction works.

Once that works, the rest becomes an engineering problem rather than a thought experiment.